

RESUMO DO MÓDULO

O que é Middleware

O Express é uma estrutura de roteamento e middlewares que possui funcionalidades mínimas por si só. Em essência, um aplicativo Express é composto por uma série de chamadas de funções de middleware que são executadas em sequência durante o processamento de uma solicitação.

Mas, afinal, o que é um middleware?

Um middleware é uma função que é executada entre a requisição do cliente e a resposta do servidor. Ele permite que você execute uma série de operações na solicitação, como:

- Autenticação: Verificar se o usuário possui permissões para acessar uma rota.
- Validação de dados: Validar os dados enviados na requisição.
- Manipulação de erros: Capturar e tratar erros antes que eles cheguem ao usuário final.

Os middlewares são peças fundamentais no Express e são utilizados para executar lógica específica em uma rota ou em várias rotas. Eles têm acesso a três objetos principais:

- req (request): Contém informações sobre a requisição enviada pelo cliente.
- res (response): Permite que você envie uma resposta para o cliente.
- next: É uma função que permite passar o controle para o próximo middleware na cadeia.

Como Utilizar Middlewares no Express?

Para adicionar um middleware em um aplicativo Express, usamos a função `app.use()`. Vamos ver um exemplo simples de um middleware que registra a data e a hora em que a solicitação foi recebida:

javascript

```
const express = require('express');

const app = express();

// Middleware para registrar a data e hora da requisição

app.use((req, res, next) => {

  console.log(`Requisição recebida em ${new Date()}`);

  next(); // Passa o controle para o próximo middleware ou rota

});

// Rota de exemplo

app.get('/users', (req, res) => {

  console.log('Caiu na rota de usuários');

  res.send('Rota de usuários');

});

// Inicializa o servidor

app.listen(3000, () => {

  console.log('Servidor iniciado na porta 3000');

});
```

Explicação do Exemplo

1. Definimos um middleware usando `app.use()`.
2. O middleware registra a data e a hora de cada requisição recebida (`console.log`).
3. Usamos `next()` para passar o controle para o próximo middleware ou rota.
4. Definimos a rota `/users`, que também executa um `console.log` quando é acessada.

5. Se acessarmos a rota /users, o console exibirá primeiro a mensagem do middleware e depois a mensagem da rota.

Por Que Usar Middlewares?

Usar middlewares é uma prática comum no desenvolvimento de aplicativos web com Express.js porque eles permitem modularizar o código e separar responsabilidades. Em vez de duplicar lógica em várias rotas, você pode definir um middleware e aplicá-lo onde for necessário.

Por exemplo, em vez de verificar a autenticação manualmente em cada rota, você pode criar um middleware que faça isso e aplicá-lo a todas as rotas que precisam ser protegidas. Isso facilita a manutenção e evita repetição de código.

Outros Exemplos de Uso de Middlewares

Middleware de Autenticação

javascript

```
const checkAuth = (req, res, next) => {  
  if (req.isAuthenticated()) {  
    next();  
  } else {  
    res.status(401).send('Você precisa estar logado para acessar esta página!');  
  }  
};
```

```
app.use('/admin', checkAuth);
```

Nesse exemplo, o middleware `checkAuth` verifica se o usuário está autenticado antes de permitir o acesso à rota `/admin`.

Middleware para Manipulação de Erros

javascript

```
app.use((err, req, res, next) => {  
  console.error(err.stack);  
  res.status(500).send('Ocorreu um erro no servidor!');  
});
```

Aqui, o middleware captura qualquer erro que ocorrer no aplicativo e envia uma resposta apropriada ao cliente.

Encadeamento de Middlewares

Você pode definir vários middlewares para a mesma rota e executá-los em ordem. Por exemplo:

javascript

```
app.use((req, res, next) => {  
  console.log('Primeiro middleware');  
  next();  
});
```

```
app.use((req, res, next) => {  
  console.log('Segundo middleware');  
  next();  
});
```

```
});
```

```
app.get('/', (req, res) => {  
  res.send('Página inicial');  
});
```

Quando você acessar a rota /, verá as mensagens no console em ordem:

- Primeiro middleware
- Segundo middleware

Isso demonstra como os middlewares são executados na sequência em que foram definidos.

Conclusão

Os middlewares são fundamentais no desenvolvimento com Express.js. Eles permitem manipular e transformar as requisições de várias formas antes que a resposta seja enviada ao cliente. O uso estratégico de middlewares permite criar aplicações mais escaláveis, robustas e seguras.
